

[← Back to Project Home](#)

Problem Statement

EGEN5208W Databases for Soft Engineers

Course: EGEN5208W Databases for Soft Engineers

Instructor: Abdelghny Orogat

Department: Department of Systems and Computer Engineering

University: Carleton University

1. Project Objective

Design and implement a **Health and Fitness Club Management System** using PostgreSQL as the database backend.

The system models the core database functionality required to manage a small fitness club, including:

- Members and their profiles
- Trainers and their availability
- Personal training sessions
- Group fitness classes
- Rooms and scheduling constraints

The goal of this project is to demonstrate sound database design, correct SQL usage, constraint enforcement, and role-based access, not to build a production-level application.

The system supports exactly three user roles:

- Members
- Trainers
- Administrative Staff

Each role has clearly defined responsibilities and limited access to the system.

2. System Requirements by User Role

This section describes the expected system behavior. Only the operations explicitly defined later in the Functional Requirements document must be implemented.

2.1 Member Requirements

Members represent clients of the fitness club.

Each member:

- Registers with basic personal information, including:
 - Full name
 - Email address (must be unique)
 - Date of birth
 - Gender
 - Contact information
- Can update their own profile information
- Can define one or more fitness goals (e.g., target weight or fitness objective)

Members may record health metrics such as weight or heart rate. Each metric entry:

- Is stored with a timestamp
- Is treated as a historical record
- Must not overwrite previous records

Members interact with the scheduling system by:

- Booking or canceling personal training sessions
- Registering for group fitness classes

The system must enforce the following business rules:

- A member cannot have overlapping bookings
- A member cannot register for a full class
- A booking cannot be confirmed if the assigned trainer or room is unavailable

Note: Trend analysis, charts, or advanced analytics are not required.

2.2 Trainer Requirements

Trainers are staff members who conduct personal training sessions and group classes.

Each trainer:

- Defines their availability using time slots
- Cannot define overlapping availability periods
- Can view a list of:
 - Their upcoming personal training sessions
 - Group classes they are assigned to

Trainers have read-only access to:

- Basic member profile information
- Relevant health metrics of members assigned to them

Trainers:

- Cannot modify member data
- Cannot access administrative or billing information

2.3 Administrative Staff Requirements

Administrative staff manage the operational aspects of the club.

Administrators are responsible for:

- Creating and managing group fitness classes
- Assigning:
 - Trainers
 - Rooms
 - Class capacity
- Preventing scheduling conflicts involving rooms or trainers

Administrators may also manage equipment records at a basic level:

- Equipment name
- Status (e.g., operational, under maintenance)

Billing and payments are simulated only:

- No real payment gateway is required
- The system must store:
 - Amount
 - Payment status
 - Date
 - Payment method (text field)

3. System-Wide Requirements

The system must:

- Enforce role-based access control
- Prevent invalid operations through:
 - Database constraints
 - Triggers
 - Application-level validation
- Preserve data integrity at all times

The database design must be:

- Normalized to Third Normal Form (3NF)
- Free of redundant or derived attributes unless justified

4. Design Decisions

You are expected to make reasonable and justified design decisions, including:

- Entity attributes
- Relationships
- Constraint definitions
- Trigger logic
- Error-handling behavior

All design decisions must be:

- Clearly reflected in the ER diagram
- Consistent with the database schema
- Justified in the accompanying documentation

You will not be penalized for reasonable design choices if they are consistent, documented, and correctly implemented.

[Next: Functional Requirements →](#)